Microelectronics Journal 156 (2025) 106542

Contents lists available at ScienceDirect

Microelectronics Journal

journal homepage: www.elsevier.com/locate/mejo

Knowledge transferring framework for cell library characterization

Zhengguang Tang [a], Cong Li [a,*], Guangxin Guo [a], Mingyu Ma [a], Hailong You [a], Xiaoling Lin [b]

[a] Xidian University, Xi'an, 710071, Shann'Xi, China
[b] China Electronic Product Reliability and Environmental Testing Research Institute, Guangzhou, 510610, Guangdong, China

ARTICLE INFO

Keywords:
Standard cell library
Delay model
Machine Learning
Transfer Learning

ABSTRACT

To evaluate digital circuit performance across PVT (process, voltage, and temperature) corners, standard cell library characterization requires costly simulations. Leveraging machine learning (ML) can improve the efficiency of this process. However, existing ML-based methods for cell library characterization often neglect the knowledge embedded across different timing arcs, leading to the need for extensive training data. In this paper, we propose a transfer learning (TL) framework to enhance timing characterization across multiple timing arcs. By quantifying the similarity among training tasks in the cell library using a fine-grained metric, our method enables rapid and accurate cell delay predictions through pre-training knowledge. Compared to conventional ML approaches, our TL framework improves both prediction accuracy and efficiency. Experimental results on 45 nm MOSFET and 14 nm FINFET technologies show significant error reductions of up to 80% and 67%, respectively.

1. Introduction

In the modern integrated circuit designs, standard cell libraries serve as the fundamental building blocks for the entire design and optimization process [1,2]. To evaluate the performance of standard cell under different work conditions, library characterization is essential. Considering traditional characterization process involves the utilization of numerous SPICE simulations, there are two challenges need to be addressed. On the one hand, industry-standard cell libraries typically comprise hundreds to thousands of cell types [3], encompassing various logical functions and drive strengths. On the other hand, as technology nodes shrink to nanometer scales, PVT increasingly affect chip performance [4–6]. This results in a significant increase in PVT corners with technology scaling, especially exceeding 1000 when considering process variations [7]. Therefore, the combination of aforementioned characterization requirements will result in an exponential increase in simulation runtime.

In recent years, many ML-based methods are introduced to accelerate the generation of cell library. The underlying ML models include random forest (RF) [8], support vector regression (SVR) [9], feed-forward neural network (FFNN) [10], polynomial ridge regression (PRR) [11–15]. In particular, existing works [8–15] focus on training predictive models for individual timing arcs within library cells. Although ML strategies enable on-the-fly generation of cell libraries, current ML-based methods do not fully explore the embedded knowledge from similar training tasks across cell library characterization, which leads to increased simulation costs for data preparation and limits their accuracy.

To address the exploration of pre-training knowledge among library cells, in this paper, we propose a knowledge transferring framework. In the framework, we first characterize various timing values for library cells. This process is repeated to encompass both 45 nm MOSFET and 14 nm FINFET technologies across various PVT corners. And then the similarity is measured by using Wasserstein distance, we select the task groups with minimum distance as knowledge transferring objects. Based on the determined task groups, feature augment-based transfer learning (TL) algorithm are adopted to explore pre-training knowledge. The key contributions are as follows:

- We use a fine-grained Wasserstein distance-based metric to assess similarity across hundreds of learning tasks, and demonstrate the positive guiding effect of this metric on TL.
- We utilize effective TL algorithm by augmenting features for ensemble regression models. Furthermore, we evaluate their effectiveness across various underlying models, including SVR, RF, and PRR. Compared to PRR, our proposed framework shows a significant reduction in prediction errors, up to 80%, and achieves up to a 1.96× reduction in required simulation costs.

2. Background and related work

* Corresponding author.
E-mail address: licong@xidian.edu.cn (C. Li).

https://doi.org/10.1016/j.mejo.2024.106542
Received 19 September 2024; Received in revised form 17 December 2024; Accepted 17 December 2024
Available online 24 December 2024
1879-2391/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

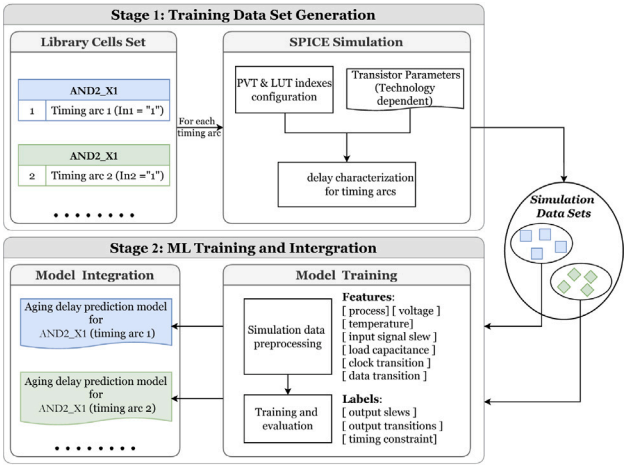

Fig. 1. Traditional flow of ML-based cell library characterization involves characterizing delay samples through SPICE simulations for each timing arc. Subsequently, prediction models are trained independently.

2.1. ML-based cell library characterization

First of all, it is important to note that cell delay is a critical factor in determining the overall performance of a circuit. Therefore, this work primarily focuses on the delay characterization of the cell library.

The standard cell library comprises various cells with multiple logical functions and drive strengths. Due to variations in cell performance under different input conditions, each cell requires simulation for different timing arcs. For each timing arc, the results are represented using look-up tables (LUTs) across different indexes, serving as fundamental components in the standard cell library. The number of components is typically in the thousands [12].

When using ML methods to characterize the standard cell library, modeling each basic component with LUT-formed data can be regarded as an individual learning task. As shown in Fig. 1, the traditional approaches involve generating LUT data for various timing arcs to create simulation datasets. Each simulation dataset is then treated as an independent learning task for fitting individual models [8–15]. To achieve accurate predictions, traditional methods rely on increasing the number of simulations to expand the training dataset.

Unlike the aforementioned works, we use TL methods to explore embedded knowledge among similar learning tasks, aiming to improve accuracy and efficiency without incurring additional simulation costs. Furthermore, our proposed TL methods is scalable to multiple underling regression models, and the effectiveness will be demonstrated in Section 4.

2.2. Transfer learning (TL)

Traditional ML techniques heavily depend on available training data, where the quality and quantity play critical roles in the training process. Limited or inaccurate training data can significantly degrade prediction performance. Generating missing data directly for the target task is challenging, but leveraging knowledge from a related domain can be valuable [16]. TL offers an effective approach to transferring knowledge between different domains or tasks, particularly beneficial for improving ML model performance when labeled data is costly [17]. Typically, the task and sample domain of interest are referred to as the ***target task and domain***, while similar tasks and sample domains are known as ***source tasks and domains***.

To effectively utilize the knowledge among thousands of learning tasks within the cell library, in this work, we utilize the feature-based TL strategy [18]. In this method, the primary focus is transferring

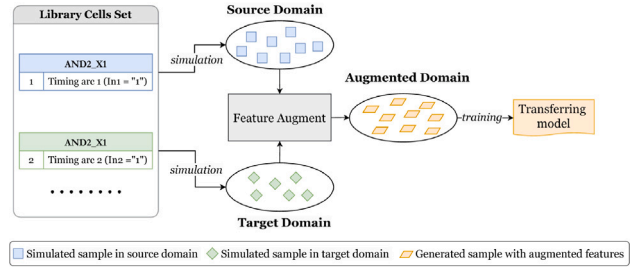

Fig. 2. Feature-based transfer learning for library cells: Feature augmentation and training.

feature representations acquired from the source domain to the target domain. This approach can also be referred to as feature augment because it enhances the original features by exploring knowledge from related domains. It helps improve the effectiveness of feature representations for the target task, leading to enhanced model performance and generalization [18]. For the standard cell library characterization, the corresponding feature-based approach is depicted in Fig. 2. Especially, the simulation data domains across similar timing arcs can be regarded as source and target domains. The augmented domain integrates information from both the source and target domains to yield high-quality predictions, without requiring additional simulation runs. Considering the numerous tasks that need to be trained in the cell library, grouping the source-target tasks is crucial for the final performance. Therefore, it is essential to quantify similarity among different tasks, which will be discussed in detail in Section 3.3.1.

3. Proposed framework

The overview of our proposed framework is illustrated in Fig. 3. As shown in this figure, three main steps are defined. In the first step, simulation samples are generated to create training and testing datasets (see Section 3.1). Next, baseline models are trained using an ensemble bagging strategy (see Section 3.2). Finally, based on task grouping methods, we apply feature-based TL algorithm to explore embedded knowledge and enhance accuracy for baseline models (see Section 3.3).

3.1. Data domains sampling

The implementation for domains sampling are presented in the left part of Fig. 3. Considering that similar timing behaviors exist across different timing arcs, in this work we chose the open-source Nangate 45 nm [19] and Nangate 14 nm [20] cell libraries with multiple sets of device models to characterize cell delays. In addition, all parameters for device and corner are provided by predictive technology model (PTM) [21].

As listed in Table 1, depending on various application scenarios, devices can be designed in high-performance (referred to as "Fast") and low-standby power (referred to as "Slow") versions. The main difference can be described as the different threshold voltages (V_{th}s) and work functions (WFs) adopted. By emulating various NMOS and PMOS process types, four process corners are symbolized as "SS", "SF", "FS", and "FF". Then, for each technology, total 48 PVT corners can be derived by combining the values from the supply voltage range and temperature list provided in Table 1.

Each timing arc in the cell library need to be characterized under every PVT corner. The *features* (corner-related parameters) and *labels* (LUT-formed cell delays) are stored in specific domains. Given a cell library for technology t containing M timing arcs, we denote the domain corresponding to the mth timing arc r_m as $\mathbb{D}(t)_{r_m}$. For training the following baseline models and performing TL optimization, the features and labels (illustrated in model training part of Fig. 1) from these domains will serve as the inputs and ground truth for prediction models.

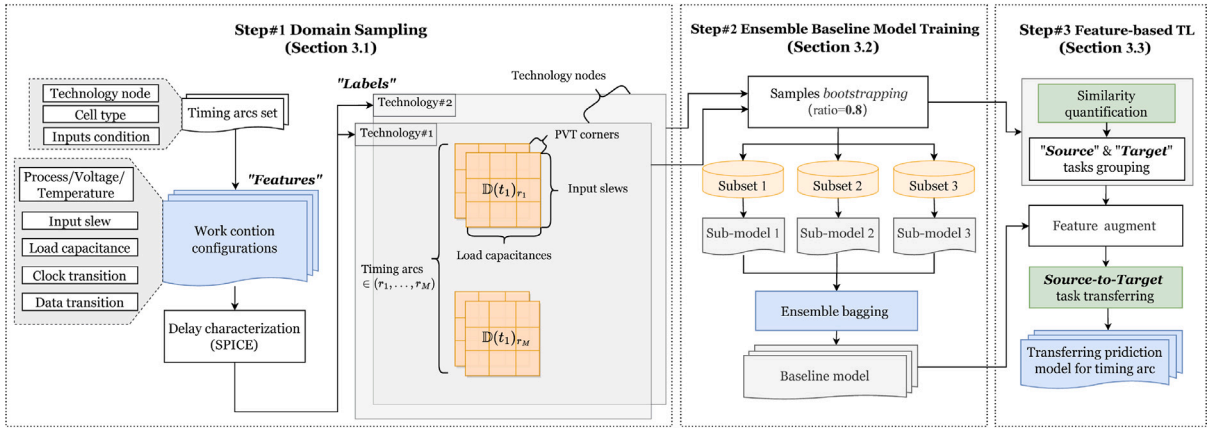

Fig. 3. Proposed knowledge transferring framework for cell library characterization.

Table 1
Parameters of Nangate 45 nm and 14 nm technology process corners.

Technology (nm)	Parameter			
	Supply voltage (V)	Temperature (°C)	Process type of NMOS	Process type of PMOS
45	[0.95, 1.1, 1.25]	[−40, 0, 25, 125]	Fast (V_{th} = 0.469 V) Slow (V_{th} = 0.623 V)	Fast (V_{th} = −0.492 V) Slow (V_{th} = −0.587 V)
14	[0.72, 0.8, 0.88]	[−40, 0, 25, 125]	Fast (WF = 4.419 eV) Slow (WF = 4.603 eV)	Fast (WF = 4.752 eV) Slow (WF = 4.574 eV)

3.2. Ensemble baseline model training

For our proposed framework shown in Fig. 3, various types of underlying ML models can be directly integrated. However, the performance of different models on various training sets varies. In order to achieve higher prediction accuracy based on few-sample learning, the following two important aspects need to be considered.

3.2.1. Baseline model construction

The baseline model refers to the model obtained by directly training on the training set using a specific underlying model. Since TL is essentially the training optimizing process of the baseline model, the quality of the baseline model significantly influences the final prediction accuracy. In our proposed framework, the ensemble bagging learning strategy is adopted to construct a high-quality baseline model.

The adopted ensemble bagging learning strategy involves training multiple instances of the same underlying model on different subsets of the training data, employing a random selection with replacement (known as bootstrapping). In addition, the sampling ratio is set to 0.8, indicating that each subset comprises 80% of samples from the training data. Finally, the baseline regression prediction is obtained by averaging the predictions of these sub-models. This approach enables the improvement of the overall performance of the model by reducing overfitting and increasing robustness. As illustrated in the step 2 in Fig. 3, balancing efficiency and accuracy considerations, 3 sub-models are separately trained for each learning task.

Furthermore, to reduce the needed training models for cell library characterization, we use multi-output modeling to predict multiple cell delays under the same input conditions in a non-linear delay model (NLDM) formatted cell library. The simultaneous predicting cell delays including `cell_fall`, `cell_rise`, `fall_transition`, `rise_transition`, `fall_constraint`, and `rise_constraint`. In summary, the baseline model for timing arc r_m in technology t can be defined as:

$$\text{Bagging} \sum_{i=1}^{3} \text{argmin}_{f_i} \mathcal{L}(f_i(\mathbf{X}), \hat{\mathbf{Y}}), \forall(\mathbf{X}, \hat{\mathbf{Y}}) \subseteq \mathbb{D}(t)_{r_m}, \tag{1}$$

Table 2
Candidate hyperparameters for underlying models.

Underlying model	Hyperparameter	Search list
RF [8]	Max_depth Min_samples_split	[3, 5, 10] [2, 5, 10]
SVR [9]	Kernel Penalty C Gamma	["rbf"] [1, 10, 100] ["scale", "auto"]
PRR [13,15]	Polynomial_degree Regularization α	[2, 3, 4,5] [1, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}]

where f_i represents the model, $\mathcal{L}$ is the loss function, $\mathbf{X}$ and $\hat{\mathbf{Y}}$ denote the training set of input features and cell delays, respectively.

3.2.2. Underlying ML model implementation

We define the underlying model as the type of ML model used for the sub-models. The learning task for cell delay prediction is a typical regression task. And different types of underlying ML models including RF [8], SVR [9], FFNN [10], and PRR [11–15] are employed to solve this regression task. Based on our experiments, configuring the optimal structure and fitting the extensive model parameters for each task using the FFNN model requires significant runtime. Furthermore, the predictive accuracy of the FFNN model is not optimal (more details can be found in Section 4.2). Therefore, three other ML models (RF, SVR, and PRR) are chosen as underlying models to demonstrate the effectiveness of our proposed framework.

In order to train and optimize hyperparameters for different underlying models, the features in the training set are initially normalized to [0, 1] using the min–max normalization method, based on the upper and lower bounds of the feature ranges listed in Table 1. Following this, grid search is performed using 5-fold cross-validation on the training set. The hyperparameters searched for our implemented underlying models are summarized in Table 2. By tuning the optimal hyperparameters for each individual underlying model, three sub-models are fitted on subsets bootstrapped from the training set. Finally, as defined in Eq. (1), all sub-models are aggregated into a baseline model for each timing arc.

3.3. Knowledge TL among similar tasks

In addition to the initial training data used for the baseline model, TL enables the incorporation of more comprehensive knowledge. This contributes to improving model prediction accuracy and reducing the required training samples.

3.3.1. Similarity quantification

As depicted in the top section of step 3 in Fig. 3, TL begins with the grouping of source and target tasks. In general, when the features or labels of grouped tasks are similar, the model can adapt more efficiently to the target task, potentially requiring less labeled data for training [17]. Therefore, assessing and maximizing similarity are essential considerations in the implementation of TL models, as they directly impact the ability of the model to generalize across different learning tasks.

The previous work [15] introduces a similarity evaluation method based on edit distance and Hamming distance. It considers that the smaller the difference among Boolean functions of timing arcs, the more similar are the learning tasks of timing arcs. However, this is an intuition-based and coarse-grained metric. For the demand of cell library characterization with a large number of different learning tasks, a theoretical and fine-grained similarity evaluation metric is required.

To address this issue, we utilize the Wasserstein distance [22] to evaluate the similarity among different learning tasks. The Wasserstein distance can be computed as:

$$W(\boldsymbol{P},\boldsymbol{Q}) = \inf_{\gamma\in\Pi(\boldsymbol{P},\boldsymbol{Q})} \mathbb{E}_{(x,y)\sim\gamma}\left[\|x-y\|\right], \tag{2}$$

where $\Pi(\boldsymbol{P},\boldsymbol{Q})$ represents the set of all possible joint distributions γ formed by combining the distributions $\boldsymbol{P}$ and $\boldsymbol{Q}$, x and y represent samples from the distributions $\boldsymbol{P}$ and $\boldsymbol{Q}$, respectively, while $\|x-y\|$ denotes the Euclidean distance between these samples. The Wasserstein distance measures the minimum cost of transforming one probability distribution in the domain $\boldsymbol{P}$ into another in the domain $\boldsymbol{Q}$. For cell library characterization, since we expect similar tasks to exhibit similar trends or characteristics for the same input features, we first randomly select 100 combinations from the complete feature space. These selected feature combinations are then used to locate the corresponding samples in each task-specific sample pool to compute the Wasserstein distance. We believe that if two different tasks exhibit smaller Wasserstein distances for the sample labels under the same feature combinations, their corresponding behaviors are more similar. This indicates that better transfer learning performance can be achieved. In addition, to address the variability in delay scales across different cells, the delay values of the selected samples are first normalized.

3.3.2. Feature-based TL algorithm

In order to effectively utilize the knowledge inherent in the selected source and target task group, we optimize the baseline models by using feature-based TL algorithm. The primary objective is to enhance the performance of models on the target task by leveraging knowledge acquired from the source task. In the proposed framework, we implement the method by augmenting with source model predictions (PRED - summarized in Algorithm 1).

For the PRED method, the source ensemble model is initially trained on the samples from the source domain $\{\mathbf{X}_S, \hat{\mathbf{Y}}_S\}$. Subsequently, each sub-model within the source ensemble model predicts samples from the target domain $\{\mathbf{X}_T, \hat{\mathbf{Y}}_T\}$. The predictions are then combined with the original target features (lines 2–4 in Algorithm 1) to generate augmented features $[\mathbf{X}_T, \mathbf{Y}_{pre_src}]$. Finally, the transfer ensemble model for target domain are derived by training sub-models on the augmented samples and applying bagging all trained sub-models (lines 5–7 in Algorithm 1).

The adopted algorithm is easily extendable and compatible with various underlying models. Last but not least, adopting this simple yet effective TL strategies requires only a small increase in training time while significantly reducing prediction errors. In addition, the increasing training time is far less than the time consumed in preparing training data through simulations.

Table 3
Tested cells from Nangate 14 nm and 45 nm libraries.

Logic function	Cell types
INV, BUF	INV_X1/X2/X4, BUF_X1/X2/X4, CLKBUF_X1/X2
AND NAND	AND2_X1/X2, AND3_X1/X2, AND4_X1/X2 NAND2_X1/X2, NAND3_X1/X2, NAND4_X1/X2
OR NOR	OR2_X1/X2, OR3_X1/X2, OR4_X1/X2 NOR2_X1/X2, NOR3_X1/X2, NOR4_X1/X2
AOI, OAI	AOI21_X1/X2, AOI22_X1/X2, OAI21_X1/X2, OAI22_X1/X2
XOR, XNOR	XOR2_X1, XNOR2_X1
FA, HA, MUX	FA_X1, HA_X1, MUX2_X1
DFF, Latch	DFFR_X1, DFFS_X1, DFFRNQ_X1, DFFSNQ_X1, DLH_X1, LHQ_X1

4. Experiment and evaluation results

4.1. Experimental setup

To validate the effectiveness of our proposed framework, we conduct experiments on 14 nm FINFET and 45 nm MOSFET cell libraries [19,20] across various PVT corners listed in Table 1. The tested cells constitute a shared subset of library cells with identical logical functions and drive strengths from both the Nangate 14 nm and Nangate 45 nm libraries. As summarized in Table 3, 48 cells are chosen for testing in each technology. According to varying input conditions across different cells in each technology, we need to model a total of 213 timing arcs to predict 799 cell delay types. In other words, our framework will be executed once for every timing arc. To prepare training data for each timing arc, we simulate cell delays based on a 7×7 index dimension. During the training process for each timing arc, 30% of the samples from the task-specific sample pool are randomly selected as the test set, while the remaining 70% constitute the complete training set. For testing with different training sizes, additional samples are randomly selected from the full training set and combined with the initial 100 samples used for distance computation to create the actual available training set. Moreover, the available training sets selected in the source domain and the target domain through the aforementioned process serve as the inputs $\mathbb{D}_S$ and $\mathbb{D}_T$ for Algorithm 1, respectively.

All experiments are tested on an AMD Ryzen 9 5900X machine (12 cores/24 threads@3.8 GHz) with a single simulation tool license. In our testing, characterizing a library typically takes about 15 min, and the overall simulation time for each process consumes approximately 12 h. Moreover, the baseline model and sub-models are constructed using the Scikit-learn [23] library, the proposed training strategies are implemented in the Python language.

In addition to assessing the prediction accuracy of trained models, we use the root mean square error (RMSE) on the test set to evaluate different strategies. As we implement every ML model in a multi-output format, the RMSE is defined as:

$$RMSE = \sum_{y\in\mathbf{Y}} \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}, \tag{3}$$

Algorithm 1 PRED transfer algorithm.

Input: sample set of source domain $\mathbb{D}_S(\mathbf{X}_S, \hat{\mathbf{Y}}_S)$
sample set of target domain $\mathbb{D}_T(\mathbf{X}_T, \hat{\mathbf{Y}}_T)$
1: $SrcOnly \leftarrow$ ensemble model training on $\{\mathbf{X}_S, \hat{\mathbf{Y}}_S\}$ ▷ see Section 3.2
2: **for** each sub-model $sm_S^i \in SrcOnly(sm_S^1, sm_S^2, sm_S^3)$ **do**
3: $\mathbf{Y}_{pre_src} \leftarrow sm_S$ predict on $\mathbf{X}_T$
4: $[\mathbf{X}_T, \mathbf{Y}_{pre_src}] \leftarrow$ concatenate $\mathbf{X}_T$ and $\mathbf{Y}_{pre_src}$
5: $sm_T^i \leftarrow$ sub-model training on $\{[\mathbf{X}_T, \mathbf{Y}_{pre_src}], \hat{\mathbf{Y}}_T\}$
6: $TrModel \leftarrow Bagging(sm_T^1, sm_T^2, sm_T^3))$
7: **return** $TrModel$

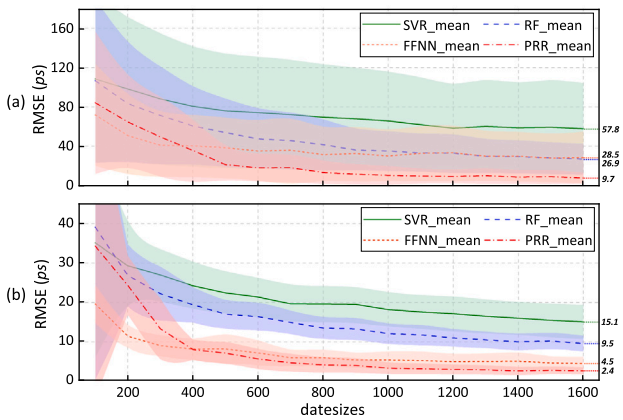

Fig. 4. Average RMSE differences with standard deviation across various underlying models for timing arcs within the (a) Nangate 45 nm and (b) Nangate 14 nm libraries.

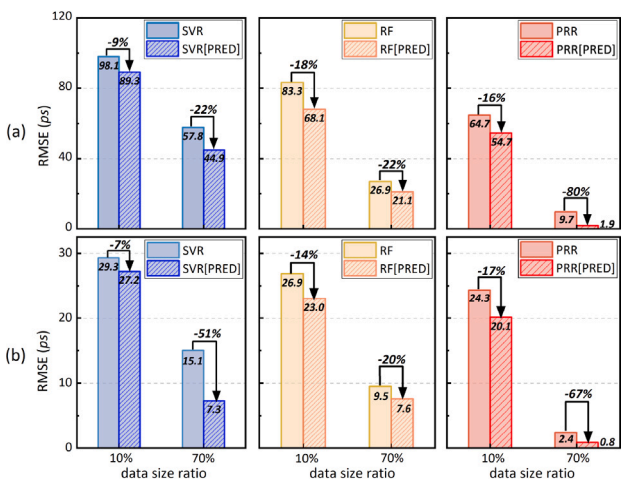

Fig. 5. Average RMSEs across various underlying models for timing arcs within (a) Nangate 45 nm and (b) Nangate 14 nm libraries.

where n is the number of test samples, y_i and $\hat{y}_i$ denote the predicted and actual label, respectively, for the ith test sample. The final RMSE is computed by summing the error values across all outputs of each model (measured in *picoseconds*). Typically, the prediction model with smaller RMSE indicates superior performance.

4.2. Delay prediction accuracy comparisons

4.2.1. Transfer across timing arcs

For each timing arc within the same technology, the source-target pair is initially determined by calculating Wasserstein distances to other timing arcs. To demonstrate the effectiveness of our proposed framework across varying scales of training sets, each timing arc is evaluated using the initial 100 samples extended in steps of 100 samples up to 70% of the samples within domains.

RMSEs across various data sizes for different baseline models are presented in Fig. 4. Summarizing all 213 tested models for 14 nm and 45 nm libraries, the baseline model with PRR [13,15] as the underlying method outperforms other prediction models, including SVR [9], RF [8], and FFNN [10], as shown in Fig. 4. This demonstrates that PRR can capture higher-order patterns between input features and cell delays by using enhanced polynomial features and regularization.

Furthermore, to demonstrate the effectiveness of our proposed TL framework on different scales of training sets, we randomly select 10% and 70% (*i.e.*, the complete training set) of samples within domains.

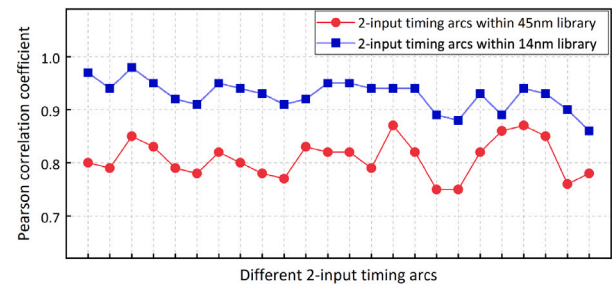

Fig. 6. Pearson correlation coefficient between Wasserstein distances vector and RMSE results vector.

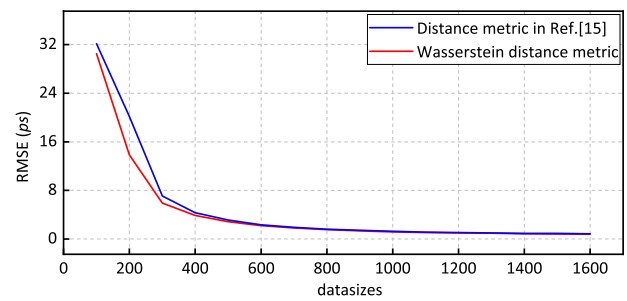

Fig. 7. Comparison of testing RMSEs between the distance metrics in [15] and the Wasserstein distance metric introduced in this work.

These samples are used to fit the baseline and TL models, with SVR, RF, and PRR as the underlying models. In addition, due to the long training time and suboptimal overall performance of FFNN, we exclude it from the TL testing scope. As shown in Fig. 5, TL models get more accurate predictions across all tested data size ratios and various underlying models. Compared with the results of baseline models, the maximum relative error reductions are achieved at the 70% dataset scale using PRR, with significant reductions of 80% and 67% for the 45 nm and 14 nm libraries, respectively.

4.2.2. Impact of similarity metrics

It is important to note that the selection of source-target groups directly influences TL performance. Therefore, we first demonstrate the effectiveness of the Wasserstein distance using all thirty timing arcs from 2-input cells within the library. Considering each timing arc within the set of 2-input timing arcs, we compute the Wasserstein distance and TL test accuracy for each pair of arcs, resulting in vectors W and R, respectively. Afterward, Pearson correlation coefficient (PCC) between W and R can be defined as:

$$PCC = \frac{Cov(W, R)}{\sigma W \cdot \sigma R}, \tag{4}$$

where $Cov(W, R)$ represents the covariance between W and R, σW and σR denote the standard deviations of W and R, respectively. Typically, a PCC greater than 0.5 is considered a strong positive correlation. This suggests that the Wasserstein distance tends to reduce the test error for TL under such a strong positive correlation. The coefficients, presented in Fig. 6, show an average value of 0.80 for the 14 nm library and 0.93 for the 45 nm library, indicating that the Wasserstein distance-based similarity metric in our framework effectively guides subsequent TL.

To validate the effectiveness of the Wasserstein distance-based similarity metric, we conduct experiments using PRR as the baseline model. The results are compared against those obtained from similar source and target tasks evaluated with edit and Hamming distances, as proposed in [15]. As shown in Fig. 7, the Wasserstein distance consistently outperforms both edit and Hamming distance metrics across all training dataset sizes. Notably, when the number of training samples is relatively small, the Wasserstein distance achieves a substantial reduction

Table 4
Testing RMSEs comparison between single training techniques and the full version.

Technology library	Methods	Logic function															Avg. RMSE of all timing arcs
		INV	BUF	AND	NAND	OR	NOR	AOI	OAI	XOR	XNOR	HA	FA	MUX	DFF	Latch	
Nangate 45 nm	PRR only	9.6	6.3	5.8	15.8	5.7	12.4	12.0	12.0	13.4	14.6	11.0	6.9	7.1	15.2	6.2	10.4
	PRR + Bagging	9.0	5.8	5.3	14.8	5.3	11.5	11.2	11.2	12.8	13.9	9.9	6.3	6.7	13.9	5.9	9.7 (7%↓)
	PRR + PRED	3.1	1.4	1.2	2.5	1.3	2.4	2.4	2.3	2.4	2.4	1.5	1.1	1.2	5.8	0.9	2.2 (79%↓)
	PRR + Bagging + PRED	**2.7**	**1.2**	**1.1**	**2.2**	**1.1**	**2.3**	**2.1**	**2.1**	**2.2**	**2.2**	**1.5**	**1.0**	**1.1**	**5.8**	**0.8**	**1.9 (82%↓)**
Nangate 14 nm	PRR only	3.1	1.5	1.5	3.1	1.6	3.8	3.1	3.1	3.3	2.7	1.9	2.1	1.8	2.1	2.7	2.6
	PRR + Bagging	2.8	1.3	1.4	2.9	1.5	3.4	2.9	2.9	2.8	1.7	1.8	2.0	1.5	1.9	2.2	2.4 (8%↓)
	PRR + PRED	1.6	0.4	0.5	1.1	0.6	1.5	1.1	1.2	1.1	0.4	0.7	0.8	0.6	0.5	0.6	1.1 (54%↓)
	PRR + Bagging + PRED	**1.4**	**0.4**	**0.4**	**1.0**	**0.4**	**1.3**	**1.1**	**1.1**	**1.1**	**0.4**	**0.4**	**0.4**	**0.4**	**0.5**	**0.6**	**0.8 (69%↓)**

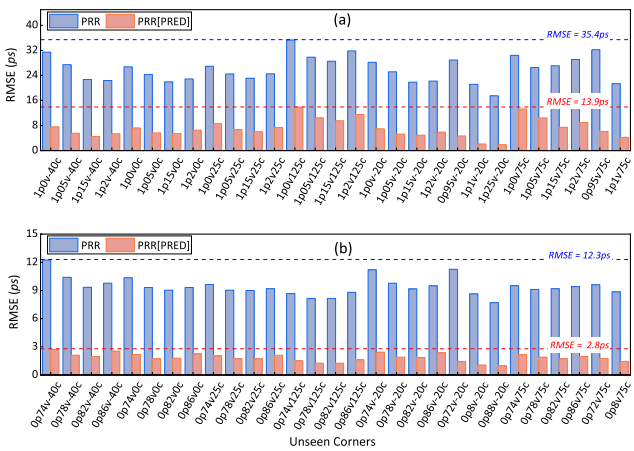

Fig. 8. Testing RMSEs comparison for unseen PVT corners within (a) Nangate 45 nm and (b) Nangate 14 nm libraries.

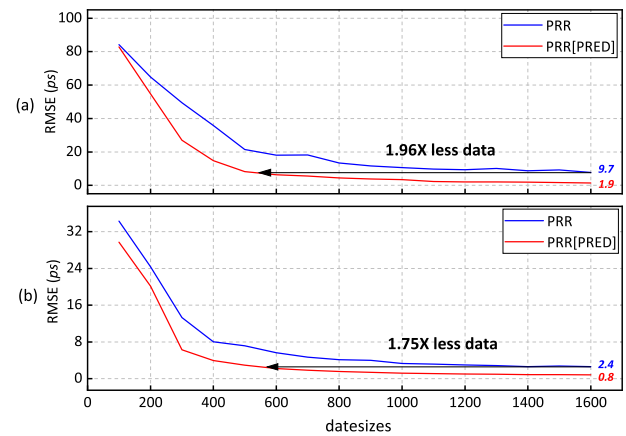

Fig. 9. Average RMSE across various datasizes for timing arcs within (a) Nangate 45 nm and (b) Nangate 14 nm libraries.

in prediction error, demonstrating its superior robustness in data-scarce scenarios.

4.2.3. Performance and generalization evaluation

In addition, we conduct an ablation study on our proposed bagging ensemble and TL strategies. As presented in Table 4, when using complete training set and testing on the baseline model with PRR, both the bagging and TL techniques independently contribute to performance improvements. Notably, the combination of all techniques yields the most substantial enhancement, achieving a remarkable RMSE reduction of 82% and 69% for the 45 nm and 14 nm libraries, respectively. Furthermore, by averaging the results of timing arcs within cells sharing the same logic function (disregarding differences in input number and drive strength) and subsequently averaging across all 213 tested timing arcs, we observe that the performance gain achieved through the TL strategy is significantly more pronounced compared to that achieved through the bagging strategy. This indicates that the TL method plays a crucial role in enhancing model prediction performance.

Furthermore, to demonstrate the generalization performance of our proposed transfer learning method, we test the trained models using the complete training set sampled from the feature space in Table 1. These models are then used to predict cell delays under unseen PVT corners. Specifically, in addition to the voltage and temperature conditions listed in Table 1, we sample additional voltage conditions with steps of 50 mV for the 45 nm library and 40 mV for the 14 nm library, resulting in unseen voltage conditions of $[1.0, 1.05, 1.15, 1.20]$ V and $[0.74, 0.78, 0.82, 0.86]$ V, respectively. Moreover, additional temperature conditions $[-20, 75]$ °C are sampled. Combined with all the voltage and temperature conditions, we construct 30 unseen PVT corners (with process fixed as SS in Table 1).

Each unseen corner is named according to its voltage and temperature, *e.g.*, 1.0 V, −40 °C as 1p0v–40c. The prediction errors of trained models for all 30 unseen corners are shown in Fig. 8. Compared to the baseline model (PRR only), our transfer learning method significantly reduces prediction errors across all unseen corners. Specifically, for the 45 nm and 14 nm libraries, the baseline model achieves a maximum RMSE of 35.4 ps and 12.7 ps, while our TL method reduces the maximum RMSE to 13.9 ps and 2.8 ps, respectively. This demonstrates that our approach significantly improves the generalization ability of cell delay prediction models on unseen corners during training process.

4.2.4. Efficiency comparisons

By leveraging an efficient TL strategy, we can reduce the number of required simulations and overall learning runtime needed to achieve a specific accuracy goal. Experimenting with PRR for optimal performance, Fig. 9 shows the trends in the amount of training samples and RMSEs for the baseline model and the TL model. To achieve the best accuracy of the baseline model, our TL strategy reduces the required simulation costs by 1.96× and 1.75× for the 45 nm and 14 nm libraries, respectively.

Moreover, based on the best RMSEs achieved by the PRR model on the largest training set, as shown in Fig. 9, we compare the total runtime of the PRR model with that of our proposed TL model. These results are summarized in Table 5. For the PRR model, the training runtime includes data processing/loading, model training, and validation. The training runtime of the TL model additionally accounts for the similarity evaluation and TL process. However, the variation in training runtime is negligible compared to the training data preparation time required for SPICE simulations. Using the trained model for inference, our approach significantly reduces the runtime for generating all 48 libraries listed in Table 1, achieving a 63% reduction for the Nangate 45 nm library and 60% for the Nangate 14 nm library.

In summary, our framework effectively utilizes embedded knowledge across various timing arcs to enhance prediction performance and reduce overall runtime.

Table 5
Runtime comparison between our model and the non-transfer learning strategy.

Technology library	Strategy	SPICE simulation (minutes)	Training (minutes)	Inference (minutes)	Total runtime (minutes)
Nangate 45 nm	PRR	470.4	**5.8**	**4.5**	480.7
	PRR [PRED]	**163.8**	8.1	6.6	**178.5 (63%↓)**
Nangate 14 nm	PRR	486.4	**6.1**	**6.3**	498.8
	PRR [PRED]	**183.1**	7.7	7.2	**198.1(60%↓)**

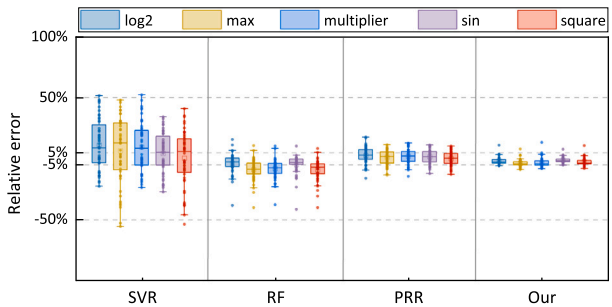

Fig. 10. Relative errors of critical path delays across various training strategies for circuits.

Table 6
The gates of critical paths and average absolute relative error for tested PVT corners.

Design	Path# Gates	SVR [9]	RF [8]	PRR [13,15]	Our model
log2	177	18.1%	5.8%	5.8%	**2.4%**
max	149	23.2%	9.6%	5.2%	**2.1%**
multiplier	205	17.5%	8.9%	5.4%	**2.5%**
sin	143	14.1%	5.3%	4.9%	**2.2%**
square	154	18.3%	9.4%	4.6%	**2.4%**
Avg.	–	18.2%	7.8%	5.2%	**2.3%**

4.3. Model inference for STA

To demonstrate the efficiency of our cell libraries for static timing analysis (STA), we use models trained on 70% of the Nangate 14 nm dataset, as discussed in Section 4.2.1, to generate libraries for all 48 PVT corners listed in Table 1. The critical path delays obtained from generated libraries are compared against the golden results derived from simulated libraries. Additionally, we evaluate the performance on five circuits from EPFL benchmarks [24]. As shown Fig. 10, when comparing with other methods such as SVR [9], RF [8], and PRR [13,15], the relative errors of critical path delays based on our framework-generated libraries are mostly concentrated within 5%.

Furthermore, the gates of critical paths in different circuits and average absolute relative error are given in the Table 6. We can observe that the prediction relative errors vary across different critical path structures, with worst case showing errors exceeding 20%. Although PRR [13,15] demonstrates overall performance better than SVR [9] and RF [8] models with an average error of 5.2%, using our proposed framework further reduces this error to 2.3%. Finally, we compare the runtime of our model inference for single cell library generation with simulation. Generating a single library takes 6.7 s, achieving a 134× speedup compared to the simulation process, which requires 15 min for characterization. This demonstrates that our framework enables on-the-fly cell library generation.

5. Conclusion

Accurate and efficient ML-based library characterization is crucial for reducing simulation costs. In this paper, we present a TL-based framework that utilize pre-training knowledge to enhance prediction accuracy and efficiency. Our framework exhibits versatility by adapting to different underlying models and achieving positive transfer effects. Experimental results show that the TL method effectively utilizes embedded knowledge from different timing arcs, leading to enhanced prediction accuracy. This enables an accurate and efficient ML-based library characterization process. Furthermore, our framework facilitates efficient STA for circuits with much lower runtime overhead. In the future, we will try to transfer among different technology nodes.

CRediT authorship contribution statement

Zhengguang Tang: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing, Visualization. **Cong Li:** Investigation, Writing – original draft, Writing – review & editing, Supervision, Project administration. **Guangxin Guo:** Investigation, Writing – review & editing. **Mingyu Ma:** Investigation, Writing – review & editing. **Hailong You:** Investigation, Writing – original draft, Writing – review & editing, Supervision, Project administration. **Xiaoling Lin:** Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Project of Science and Technology on Reliability Physics and Application Technology of Electronic Component Laboratory, China under Grant 6142806230302.

Data availability

The data that has been used is confidential.

References

[1] V. Vashishtha, L.T. Clark, Asap5: A predictive pdk for the 5 nm node, Microelectron. J. 126 (2022) 105481, http://dx.doi.org/10.1016/j.mejo.2022.105481.
[2] Zejia Lyu, Jizhong Shen, An efficient algorithm for estimating gate-level power consumption in large-scale integrated circuits, Microelectron. J. 146 (2024) 106143, http://dx.doi.org/10.1016/j.mejo.2024.106143.
[3] P. Sharma, P. Jain, B.P. Das, An optimal device sizing for a performance-driven and area-efficient subthreshold cell library for iot applications, Microelectron. J. 92 (2019) 104613, http://dx.doi.org/10.1016/j.mejo.2019.104613.
[4] W.W. Xing, L. Wang, Z. Wang, Z. Shi, N. Xu, Y. Cheng, W. Zhao, Multi-corner timing analysis acceleration for iterative physical design of ics, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 1 (2024) http://dx.doi.org/10.1109/TCAD.2024.3361401.
[5] Yan Liu, Yan-Hua Ma, Yu-Chun Chang, Timing analysis algorithm using a neural network under pvt variations, Microelectron. J. 125 (2022) 105480, http://dx.doi.org/10.1016/j.mejo.2022.105480.
[6] Xinsheng Wang, Yanhong Song, Mingyan Yu, Comprehensive probability method of buffer insertion based on gaussian fitting under process variation condition, Microelectron. J. 134 (2023) 105728, http://dx.doi.org/10.1016/j.mejo.2023.105728.
[7] S. Dongaonkar, S.P. Mudanai, M.D. Giles, From process corners to statistical circuit design methodology: Opportunities and challenges, IEEE Trans. Electron Devices 66 (2019) 19–27, http://dx.doi.org/10.1109/TED.2018.2860929.
[8] S. Bian, M. Shintani, M. Hiromoto, T. Sato, Lsta: Learning-based static timing analysis for high-dimensional correlated on-chip variations, in: Proceedings of the 54th Annual Design Automation Conference 2017, 2017, pp. 1–6, http://dx.doi.org/10.1145/3061639.3062280.
[9] A. Vijayan, K. Chakrabarty, M.B. Tahoori, Machine learning-based aging analysis, Machine Learn. VLSI Comput.-Aided Des. 26 (2019) 5–289, http://dx.doi.org/10.1007/978-3-030-04666-8_9.
[10] S.M. Ebrahimipour, B. Ghavami, H. Mousavi, M. Raji, Z. Fang, L. Shannon, Aadam: A fast, accurate, and versatile aging-aware cell library delay model using feed-forward neural network, in: Proceedings of the 39th International Conference on Computer-Aided Design, 2020, pp. 1–9, http://dx.doi.org/10.1145/3400302.3415605.

[11] F. Klemme, Y. Chauhan, J. Henkel, H. Amrouch, Cell library characterization using machine learning for design technology co-optimization, in: 2020 IEEE/ACM International Conference on Computer Aided Design, ICCAD, 2020, pp. 1–9, http://dx.doi.org/10.1145/3400302.3415713.

[12] F. Klemme, H. Amrouch, Machine learning for on-the-fly reliability-aware cell library characterization, IEEE Trans. Circuits Syst. I. Regul. Pap. 68 (2021) 2569–2579, http://dx.doi.org/10.1109/TCSI.2021.3069664.

[13] F. Klemme, H. Amrouch, Efficient learning strategies for machine learning-based characterization of aging-aware cell libraries, IEEE Trans. Circuits Syst. I. Regul. Pap. 69 (2022) 5233–5246, http://dx.doi.org/10.1109/TCSI.2022.3201431.

[14] X. Zhang, Z. Zhang, Y. Lin, Z. Ji, R. Wang, R. Huang, Efficient aging-aware standard cell library characterization based on sensitivity analysis, IEEE Trans. Circuits Syst. II 70 (2023) 721–725, http://dx.doi.org/10.1109/TCSII.2022.3212123.

[15] Z. Tang, C. Li, H. You, Z. Di, L. Zhang, X. Liu, Y. Wang, Y. Dai, G. Bai, Semi-supervised transfer learning framework for aging-aware library characterization, IEEE Trans. Circuits Syst. II 71 (2024) 1156–1160, http://dx.doi.org/10.1109/TCSII.2023.3323384.

[16] G. Yang, C. Jiang, R. Han, J. Tan, F. Liu, Reducing parameter estimation error of behavioral modeling and digital predistortion via transfer learning for rf power amplifiers, IEEE Trans. Microw. Theory Tech. 71 (2023) 4787–4799, http://dx.doi.org/10.1109/TMTT.2023.3267117.

[17] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, Q. He, A comprehensive survey on transfer learning, Proc. IEEE 109 (2021) 43–76, http://dx.doi.org/10.1109/JPROC.2020.3004555.

[18] H. Daumé III, Frustratingly easy domain adaptation, 2009, http://dx.doi.org/10.48550/arXiv.0907.1815, arXiv preprint arXiv:0907.1815.

[19] Si2, Nangate open cell library, 2019, URL https://si2.org/open-cell-library/. (28 Accessed: March 2024).

[20] Silvaco, Silvaco and si2 release unique, free 15 nm open-source digital cell library, 2019, URL https://silvaco.com/news/silvaco-and-si2-release-unique-free-15nm-open-source-digital-cell-library. (28 Accessed: March 2024).

[21] W. Zhao, Y. Cao, Predictive technology model for nano-cmos design exploration, ACM J. Emerg. Technol. Comput. Syst. (JETC) 3 (2007) 1–es, http://dx.doi.org/10.1145/1229175.1229176.

[22] J. Shen, Y. Qu, W. Zhang, Y. Yu, Wasserstein distance guided representation learning for domain adaptation, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2018, http://dx.doi.org/10.1609/aaai.v32i1.11784.

[23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in python, J. Mach. Learning Res. 12 (2011) 2825–2830, http://dx.doi.org/10.48550/arXiv.1201.0490.

[24] L. Amarú, P.E. Gaillardon, G. De Micheli, The epfl combinational benchmark suite, in: Proceedings of the 24th International Workshop on Logic & Synthesis, IWLS, 2015, URL https://infoscience.epfl.ch/handle/20.500.14299/113476.